

AI Lab-3

BS DS FALL 2024

Introduction to Genetic Algorithms

Genetic Algorithms are search and optimization techniques inspired by the principles of natural selection and genetics. These algorithms evolve a population of candidate solutions over multiple generations to find better solutions to a problem.

Genetic Algorithms are especially useful when:

- the search space is very large
- traditional methods are slow or ineffective
- approximate but good solutions are acceptable

Instead of trying every possible solution, a GA starts with random solutions and gradually improves them using biological concepts such as:

- selection
- crossover
- mutation

Key Concepts

1. **Population:** A population is a set of possible solutions to the problem.
2. **Chromosome:** A chromosome represents one complete candidate solution.
3. **Gene:** A gene is one part of a chromosome. In this lab, each gene will indicate whether an item is included in the knapsack or not.
4. **Fitness Function:** The fitness function measures how good a solution is.
5. **Selection:** Selection chooses the better chromosomes as parents for the next generation.
6. **Crossover:** Crossover combines two parent chromosomes to produce offspring.
7. **Mutation:** Mutation randomly changes a gene to maintain diversity in the population.

Task 01: Genetic Algorithm for the Knapsack Problem

Problem Statement

You are given a set of items. Each item has:

- a weight
- a value

You are also given a knapsack with a maximum capacity.

Your goal is to select items such that:

- the total weight does not exceed the knapsack capacity
- the total value is maximized

Chromosome Representation

Use a binary chromosome:

- 1 means the item is selected
- 0 means the item is not selected

If there are 30 items, then each chromosome must also have length 30.

Example:

[1, 0, 1, 1, 0, ..., 1]

Dataset

Use an item set of 30 items.

You may use the following default dataset:

```
items = [  
  (2, 12), (1, 10), (3, 20), (2, 15), (4, 25),  
  (5, 30), (7, 42), (6, 35), (3, 18), (2, 14),  
  (8, 50), (9, 55), (4, 24), (5, 28), (1, 8),  
  (6, 33), (7, 40), (3, 16), (2, 11), (4, 22),  
  (10, 60), (9, 52), (5, 29), (6, 34), (2, 13),  
  (1, 7), (8, 48), (7, 39), (3, 19), (4, 23)  
]
```

Suggested knapsack capacity:

MAX_WEIGHT = 60

Additional Requirements

The program should allow the user to enter:

- the number of items, or directly input item weights and values
- the knapsack capacity
- the selection method:
 - roulette wheel selection
 - tournament selection

A default dataset may be provided, but user input should also be supported.

Code Template:

```
import random
items = [
(2, 12), (1, 10), (3, 20), (2, 15), (4, 25), (5, 30), (7, 42), (6, 35), (3, 18), (2,
14), (8, 50), (9, 55), (4, 24), (5, 28), (1, 8), (6, 33), (7, 40), (3, 16), (2, 11),
(4, 22), (10, 60), (9, 52), (5, 29), (6, 34), (2, 13), (1, 7), (8, 48), (7, 39), (3,
19), (4, 23)
]
MAX_WEIGHT = 60
POPULATION_SIZE = 30
MUTATION_RATE = 0.05
GENERATIONS = 50
TOURNAMENT_SIZE = 3
# =====
# User Input Option
# =====
def get_user_input():
pass

# =====
# Initialize Population
# =====
def initialize_population(size, num_items):
pass

# =====
# Fitness Function
# =====
def fitness(chromosome, items, max_weight):
pass

# =====
# Weight Calculator
# =====
def calculate_weight(chromosome, items):
pass
```

```

# =====
# Roulette Wheel Selection
# =====
def roulette_wheel_selection(population, items, max_weight):
pass

# =====
# Tournament Selection
# =====
def tournament_selection(population, items, max_weight, k=3):
pass

# =====
# Selection Dispatcher
# =====
def select_parent(population, items, max_weight, method):
pass

# =====
# Crossover
# =====
def crossover(parent1, parent2):
pass

# =====
# Mutation
# =====
def mutate(chromosome, mutation_rate):
pass

# =====
# Genetic Algorithm
# =====
def genetic_algorithm(items, max_weight, selection_method):
pass

# =====
# Display Solution
# =====
def display_solution(solution, items):
pass

def main():
pass

if __name__ == "__main__":
main()

```

Implementation Steps

1.1 Define the Item List

Create a list of about 30 items with weights and values. Also allow the user to enter a custom item list and knapsack capacity.

1.2 Initialize the Population

Generate a random population of binary chromosomes of length equal to the number of items.

1.3 Implement the Fitness Function

Write a function that:

- computes total weight
- computes total value
- returns 0 if total weight exceeds the capacity
- otherwise returns the total value

1.4 Implement the Selection Methods

Implement both:

- roulette wheel selection
- tournament selection

The user should be able to choose the selection method at runtime.

1.5 Implement Crossover

Use single-point crossover.

1.6 Implement Mutation

Flip genes randomly using a mutation probability.

1.7 Run the Genetic Algorithm

Run the algorithm for a fixed number of generations and track the best solution.

1.8 Display the Final Solution

Print:

- best chromosome
- total fitness
- selected items
- total weight
- total value
- selection method used

Task 02: Genetic Algorithm for the Travelling Salesman Problem (TSP)

Problem Statement

In the Travelling Salesman Problem, a salesman must visit each city exactly once and return to the starting city.

The objective is to find a route with the minimum total distance.

This is different from knapsack:

- knapsack uses a binary chromosome
- TSP uses a permutation chromosome

Chromosome Representation

A chromosome is a sequence of city indices showing the visiting order.

Example:

[0, 3, 1, 4, 2]

This means:

- start at city 0
- then visit city 3
- then city 1
- then city 4
- then city 2
- then return to city 0

Sample City Coordinates

cities = {

0: (2, 3),

1: (5, 4),

2: (1, 7),

3: (6, 8),

4: (9, 2),

5: (4, 6),

6: (8, 7),

7: (3, 9)

}

Code Template:

```
import random
import math

cities = {
0: (2, 3),
1: (5, 4),
2: (1, 7),
3: (6, 8),
4: (9, 2),
5: (4, 6),
6: (8, 7),
7: (3, 9)
}

POPULATION_SIZE = 30
GENERATIONS = 100
MUTATION_RATE = 0.05
TOURNAMENT_SIZE = 3

def distance(city1, city2):
pass

def total_distance(route, cities):
pass

def fitness(route, cities):
pass

def initialize_population(size, num_cities):
pass

def roulette_wheel_selection(population, cities):
pass

def tournament_selection(population, cities, k=3):
pass

def select_parent(population, cities, method):
pass

def crossover(parent1, parent2):
pass

def mutate(route, mutation_rate):
pass
```

```
def genetic_algorithm(cities, selection_method):  
    pass  
  
def display_solution(best_route, cities):  
    pass  
  
def main():  
    pass  
  
if __name__ == "__main__":  
    main()
```

Implementation Steps

2.1 Define the Cities

Create a list or dictionary of city coordinates.

Optionally allow the user to enter the number of cities and their coordinates.

2.2 Initialize the Population

Generate a random population of valid routes.

Each chromosome must be a permutation of all city indices.

2.3 Implement the Distance Function

Write a function to compute the total route distance, including the return to the starting city.

2.4 Implement the Fitness Function

Since GA usually maximizes fitness, define fitness as:

$\text{fitness} = 1 / \text{total_distance}$

This ensures shorter routes have higher fitness.

2.5 Implement Two Selection Methods

Students must implement:

- roulette wheel selection
- tournament selection

The user should choose the selection method.

2.6 Implement Crossover

Use a crossover suitable for permutations, such as:

- Order Crossover (OX), or
- Partially Matched Crossover (PMX)

Single-point crossover is not suitable for TSP because it can create invalid routes with repeated cities.

2.7 Implement Mutation

Use a mutation method suitable for TSP, such as:

- swap mutation
- inversion mutation

2.8 Run the Genetic Algorithm

Run the GA for several generations and track the shortest route found.

2.9 Display the Best Route

Print:

- best route
- total distance
- fitness
- selection method used